Instituto Infnet

Análise e Segurança de Agentes de IA

Desenvolvimento Seguro de Aplicações Web

2026T3

Yohann M G Guedes

Assessment - API de agendamento de consultas

Vídeo de apresentação (YouTube, não listado): https://youtu.be/7MsXzOAF8zM

Visão geral

API REST de agendamento de consultas (FastAPI) para uma rede de clínicas. Atende três clientes: frontend JSON de pacientes e profissionais, página HTML interna da recepção e integração máquina a máquina de um laboratório parceiro. Trata dado de saúde, categoria sensível da LGPD (art. 5º, II e art. 11), o que orientou todas as decisões.

Item	**Onde**
Código	`app/` (`routes`, `models`, `database`, camada `auth`), `main.py`
Testes	`tests/` (174 testes pytest)
Pipeline	`.github/workflows/security.yml`, `ci/security_gate.py`, `ci/auditar_openapi.py`
Evidências	`evidencias/` (saídas de requisições, telas, relatórios de SAST, SCA, ZAP e logs do GitHub Actions)
Diagramas	`docs/diagramas/`

Exercício 1 - Fundação da API

- Ambiente isolado com `python -m venv venv` e dependências fixadas em `requirements.txt` e `requirements-dev.txt`. Fixar versões garante que o pip-audit do Exercício 12 audite exatamente o que vai para produção.

- Estrutura modular: `app/routes` (um `APIRouter` por recurso), `app/models.py` (contratos), `app/db_models.py` (tabelas), `app/database.py` (repositório) e `app/auth/` (autenticação separada). Nenhuma regra de segurança aparece em mais de um lugar.
- Recurso `consultas` completo: `GET /consultas`, `GET /consultas/agenda`, `POST`, `GET/PUT/DELETE /consultas/{id}`. O cancelamento é lógico, para não destruir histórico clínico.
- Primeiro teste: `tests/test_consultas.py::test_profissional_cria_consulta_e_ela_aparece_na_listagem`.

Evidência: `evidencias/ex01-fundacao.txt` (venv ativo, `pip freeze`, pytest).

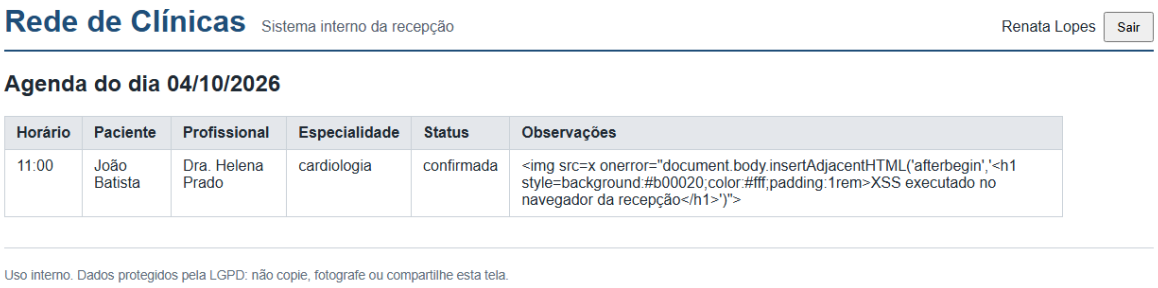
Exercício 2 - Exposição de dados e templates seguros

- A tabela `Consulta` guarda campos internos (`token_auditoria`, `criado_por_id`, `criado_em`, `atualizado_em`, `ip_origem`); a API devolve só os contratos de saída, por `response_model`:

Consumidor	Contrato	Recorte
Profissional e paciente	`ConsultaPublic`	Sem campos internos
Recepção	`ConsultaAgenda`	Sem campos internos e sem o motivo clínico (princípio da necessidade, LGPD art. 6º, III)
Laboratório	`HorarioDisponivel`	Nenhum dado de paciente
Cadastro de paciente	`PacientePublic`	CPF mascarado (`***.018.159-**`)

- Página da recepção com herança de templates: `templates/base.html` e `templates/agenda.html` (`{% extends %}`). O autoescape do Jinja2 codifica as

observações, e nenhum template usa `|safe` com dado de usuário. O CSS fica em arquivo estático para a CSP não precisar de `'unsafe-inline'`.



Rede de Clínicas Sistema interno da recepção

Renata Lopes Sair

Agenda do dia 04/10/2026

Horário	Paciente	Profissional	Especialidade	Status	Observações
11:00	João Batista	Dra. Helena Prado	cardiologia	confirmada	<img src=x onerror="document.body.insertAdjacentHTML('afterbegin','<h1 style=background:#b00020;color:#fff;padding:1rem>XSS executado no navegador da recepção</h1>')">

Uso interno. Dados protegidos pela LGPD: não copie, fotografe ou compartilhe esta tela.

Agenda corrigida: o payload aparece como texto

Evidência: `evidencias/ex02-response-model-e-template.txt`.

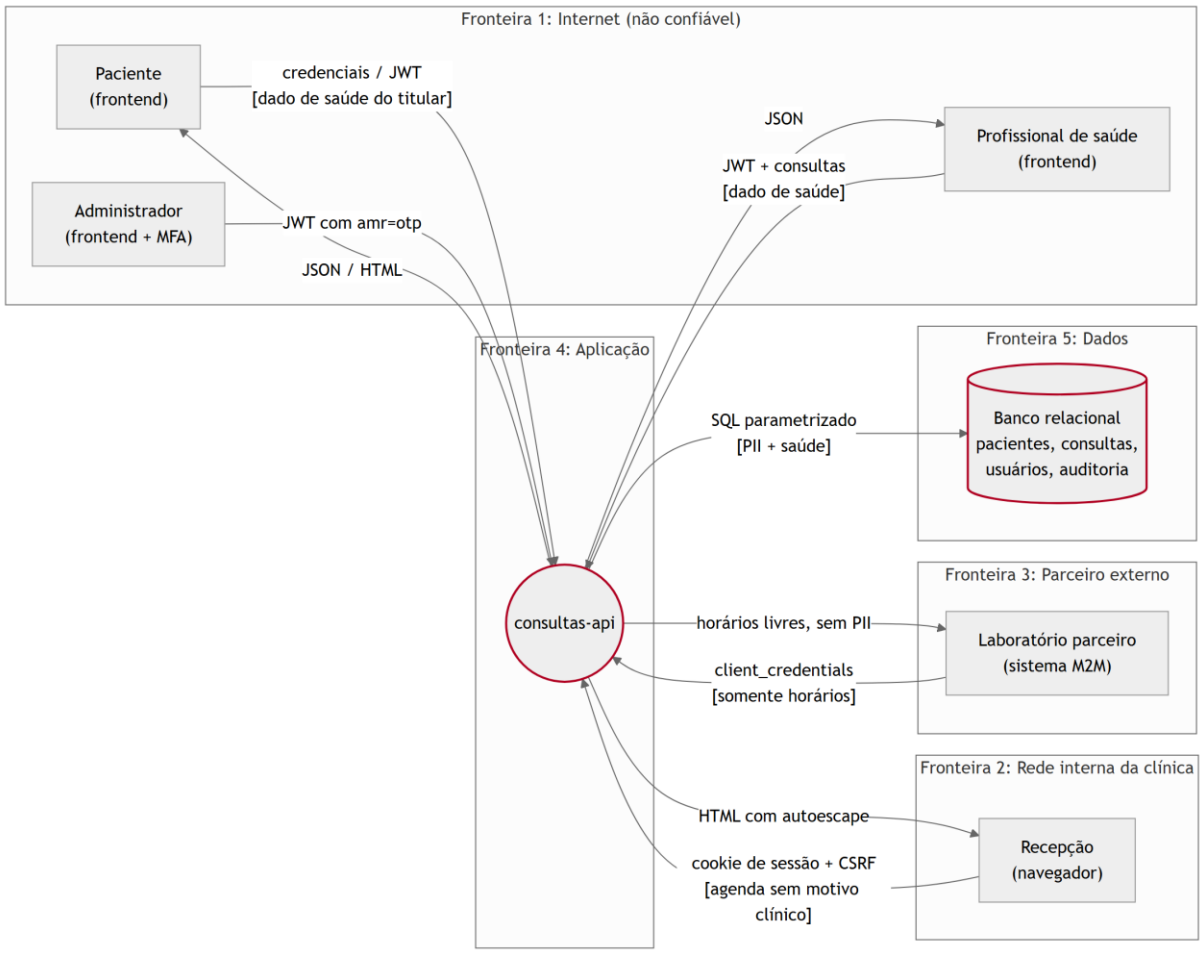
Exercício 3 - CIA, frameworks e DFD

Confidencialidade tem o maior peso: vazamento de dado de saúde é irreversível e gera comunicação obrigatória à ANPD (art. 48).

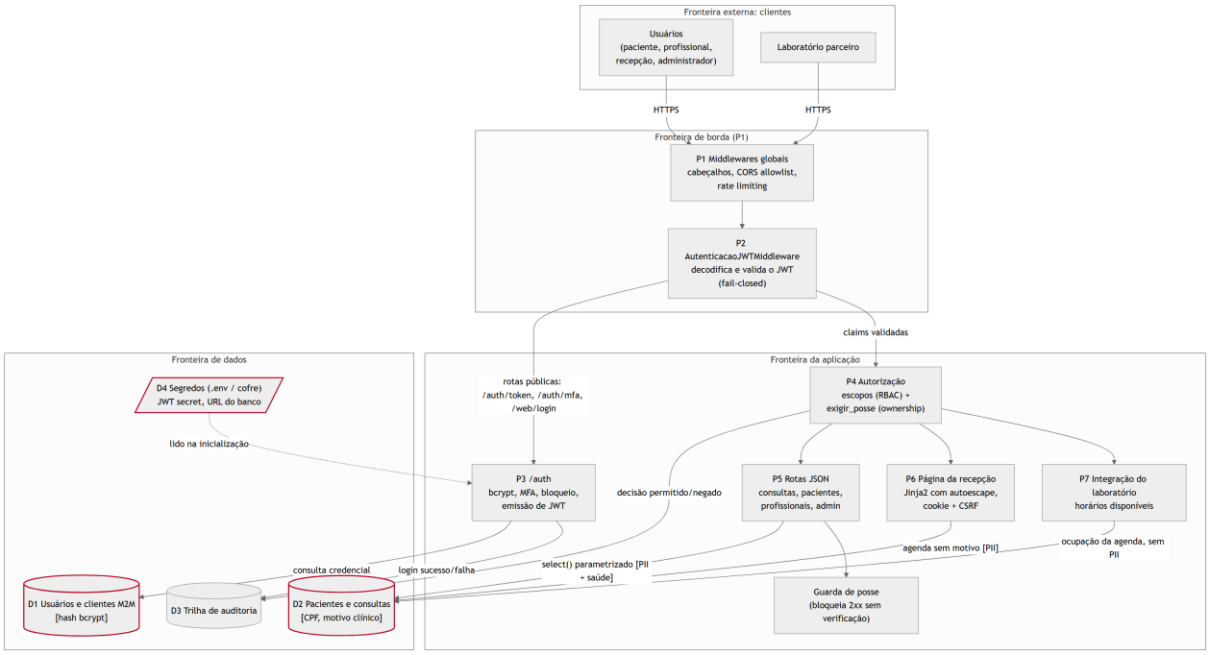
Propriedade	Ameaça principal	Controle
Confidencialidade	Paciente lendo consulta de outro (BOLA); campos internos na resposta	`exigir_posse`, `response_model`, CPF mascarado

Integridade	Alteração de consulta alheia; mass assignment; XSS; JWT adulterado	`consulta_alteravel`, `extra="forbid"`, autoescape, JWT com algoritmo fixo
Disponibilidade	Força bruta e flood	Rate limiting por tipo de rota, bloqueio por identidade

Framework	**Item**	**Controle implementado**
OWASP Top 10 2021	A01, A02, A03, A04, A05, A07, A09	Posse do objeto, bcrypt, consultas parametrizadas, `extra="forbid"`, cabeçalhos, MFA, trilha de auditoria
OWASP API Top 10 2023	API1 BOLA, API3 BOPLA, API5 BFLA	`exigir_posse`, contratos de saída, `exigir_administrador`
NIST SSDF	PO.1, PS.1, PW.5, PW.7, RV.1	Threat model com testes T-xx, segredos fora do código, validação centralizada, SAST/DAST, gate
MITRE ATT&CK	T1110, T1078, T1190, T1552	Rate limit, MFA, validação, segredo obrigatório no ambiente



DFD nível 0: fronteiras de confiança e fluxos de dado de saúde



DFD nível 1: processos da aplicação

As cinco fronteiras de confiança são: Internet, rede da clínica, parceiro, aplicação e dados. Toda travessia tem um controle: TLS e CORS na borda, cookie HttpOnly e CSRF na recepção, client_credentials para o parceiro, middleware JWT na entrada da aplicação e SQL parametrizado no acesso ao banco.

Exercício 4 - Threat model (STRIDE)

Ativos: dado de saúde (A1), dado identificador do paciente (A2), credenciais (A3), segredo JWT e URL do banco (A4), integridade da agenda (A5), trilha de auditoria (A6), disponibilidade (A7).

Superfícies de ataque: `/auth/token` e `/auth/mfa` (públicas), rotas de consultas e pacientes, rota do laboratório, rotas administrativas, página da recepção, `/docs` e configuração.

Misuse cases:

ID	Caso	Ameaças

MC-01	Paciente troca o id na URL e lê consultas de outros	T-01
MC-03	Token vazado do laboratório usado para ler pacientes	T-04
MC-04	Força bruta no login da recepção	T-05
MC-05	`<script>` nas observações sequestrando a sessão da recepção	T-10, T-19
MC-06	SQL injection na busca de pacientes	T-09
MC-07	`profissional_id` no corpo para agendar em nome de outro	T-11
MC-08	Recepcionista tentando ver o motivo clínico	T-12

STRIDE por componente:

Componente	S	T	R	I	D	E
API de consultas	Token forjado (T-06)	Mass assignment (T-11)	Cancelamento negado (T-15)	BOLA, exposição excessiva (T-01, T-02, T-12)	Flood (T-16)	Paciente alterando consulta (T-20)
Autenticação	Força bruta, admin sem MFA (T-05, T-08)	`alg=none` (T-06)	Login sem registro (T-15)	Segredo no código (T-17)	Bloqueio como arma	Escopo além do papel (T-03, T-07)
Persistência	Credencial vazada (T-17)	SQL injection (T-09)	Alteração sem rastro (T-15)	UNION lendo hashes (T-09)	Consulta sem limite (T-16)	Escrita em campo interno (T-11)
Página da recepção	Roubo de cookie, CSRF (T-19)	XSS armazenado (T-10)	Acesso sem registro (T-15)	Clickjacking, cache (T-13)	Script travando a página	Script agindo com a sessão (T-10)

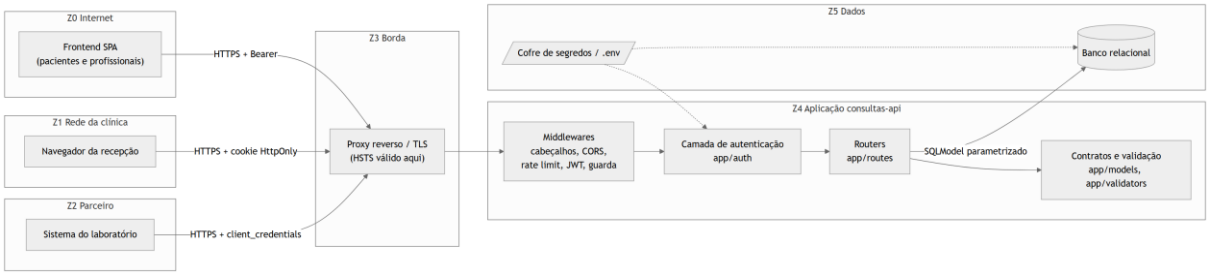
Integração do laboratório	Segredo vazado (T-04)	Parâmetro manipulado (T-09)	Chamadas sem registro (T-15)	PII na resposta (T-04)	Varredura de datas (T-16)	Escopo fora do contrato (T-04)

Catálogo consolidado (T-xx → mitigação → teste):

ID	Ameaça	OWASP	Mitigação	Teste
T-01	BOLA em consultas	A01	`exigir_posse` + guarda de posse	`test_t01_*`
T-02	BOLA em pacientes	A01	`paciente_legivel`, busca filtrada	`test_t02_*`
T-03	Acesso a rotas admin	A01	Escopo `admin` + `exigir_administrador`	`test_t03_*`
T-04	Token do laboratório além do contrato	A01	Recorte de escopo, `aud` próprio	`test_t04_*`
T-05	Força bruta	A07	Rate limit 5/min + bloqueio por identidade	`test_t05_*`
T-06	JWT forjado	A02/A07	Algoritmo fixo, claims obrigatórias	`test_t06_*`
T-07	Token expirado ou de outra audiência	A07	`exp` curto, `aud` por tipo	`test_t07_*`
T-08	Admin sem MFA	A07	Desafio MFA, claim `mfa` exigida	`test_t08_*`
T-09	SQL injection	A03	Whitelist + `select()` parametrizado	`test_t09_*`
T-10	XSS armazenado	A03	Autoescape + CSP	`test_pagina_da_agenda_*`
T-11	Mass assignment	A04	`extra="forbid"`	`test_t11_*`
T-12	Exposição excessiva	A04	Contratos por consumidor	`test_resposta_*`

T-13	Cabeçalhos ausentes	A05	Middleware de cabeçalhos	`test_t13_*`
T-14	CORS permissivo	A05	Allowlist, wildcard recusado	`test_t14_*`
T-15	Repúdio	A09	Trilha de auditoria	`test_t15_*`
T-16	DoS por volume	A04	Rate limit, limites de entrada	`test_t05_*`
T-17	Credencial no código	A07	`BaseSettings`, `SecretStr`	`test_t17_*`
T-18	Dependência vulnerável	A06	pip-audit, Dependabot	pipeline
T-19	Sessão e CSRF da recepção	A01/A07	Cookie HttpOnly/Strict, token CSRF	`test_t19_*`
T-20	Alteração por não dono	A01	`consulta_alteravel`	`test_t20_*`
T-21	Enumeração de usuários	A07	Mensagem única, bcrypt contra hash fictício	`test_t21_*`
T-22	Documentação exposta	A05	`CLINICA_DOCS_ATIVOS=false`	`test_t22_*`

Exercício 5 - Arquitetura e vetores de ataque



Partições do sistema

Partições: Z0 Internet, Z1 rede da clínica, Z2 parceiro, Z3 borda (TLS), Z4 aplicação, Z5 dados. Dentro de Z4, a camada `app/auth` é a única que decide identidade e permissão; as rotas só declaram o que exigem. Ordem dos middlewares: cabeçalhos → CORS → rate limit → JWT → guarda de posse → rota.

Eixo	**Vetores**	**Resposta**	
Design	Autorização só por papel; mesmo token para pessoa e parceiro; recepção com dado clínico; admin só com senha	RBAC + posse do objeto; audiências distintas; contrato por consumidor; MFA	
Implementação	SQL concatenado; ``	safe`; modelo aceitando campo extra; JWT sem exp`; mensagem de login reveladora	`select()`, autoescape, `extra="forbid"`, claims obrigatórias, mensagem única
Infraestrutura	Sem TLS; CORS `*`; força bruta; segredo no repositório; dependência vulnerável; `/docs` público	HSTS, allowlist, rate limit, `.env`, pip-audit, docs desligável	

Exercício 6 - Autenticação e autorização

- `OAuth2PasswordBearer` em `app/auth/dependencies.py`; senhas com bcrypt custo 12 (`app/auth/security.py`). Senha acima de 72 bytes é recusada em vez de truncada. E-mail inexistente ainda executa o bcrypt contra um hash fictício, para não revelar contas pelo tempo de resposta.
- JWT HS256 com `iss`, `aud`, `sub`, `tipo`, `papel`, `scope`, `amr`, `mfa`, `exp` (15 min) e `jti`; algoritmo fixado e claims obrigatórias. O usuário é recarregado do banco a cada requisição, então conta desativada perde o acesso na hora.

- MFA simulado para o administrador: a senha devolve um desafio; o código de 6 dígitos (gerado com `secrets`) vale 5 minutos, aceita 3 tentativas e é de uso único. O token resultante traz `mfa=true`, exigido pelas rotas administrativas.
- Sessão da recepção em cookie `HttpOnly`, `Secure`, `SameSite=Strict`, `Path=/web`, 15 minutos, com escopo único `agenda:ler`.

Modelo de autorização: RBAC + autorização por recurso. RBAC sozinho não resolve "profissional só vê os próprios pacientes", porque os dois profissionais têm o mesmo papel. ABAC exigiria um motor de políticas difícil de auditar para poucas regras estáveis. Por isso o papel define os escopos (`app/auth/permissoes.py`) e a fábrica `exigir_posse` confere o dono do objeto (profissional responsável, paciente titular ou administrador).

Escopo	Recepção	Profissional	Paciente	Admin	Laboratório
`consultas:ler` / `consultas:escrever`		x / x	x /	x / x	
`agenda:ler`	x			x	
`pacientes:ler` / `pacientes:criar`	x / x	x /		x / x	
`admin`				x	
`horarios:ler`					x

Teste pedido:
`test_t03_usuario_sem_papel_de_administrador_nao_acessa_rota_administrativa` (profissional, recepção e paciente recebem 403 nas rotas admin).

Evidência: `evidencias/ex06-autenticacao-autorizacao.txt`.

Exercício 7 - Escopos e integração M2M

Fluxo escolhido: client_credentials, o fluxo do OAuth 2.0 para comunicação máquina a máquina, sem usuário. Authorization Code pressupõe uma pessoa autorizando no navegador, e o fluxo password obrigaria a criar um usuário fictício. O segredo do laboratório fica no banco como hash bcrypt.

A limitação do contrato é técnica, não só contratual:

1. O emissor concede só a interseção entre o pedido e o contrato (`horarios:ler`); pedido só fora do contrato devolve 400.
2. O token tem `aud=consultas-api:laboratorio` e `tipo=servico`; combinação incoerente é recusada.
3. A rota do laboratório exige token de serviço, e as demais exigem escopos que ele não tem.
4. A resposta `HorarioDisponivel` não tem dado de paciente: mesmo com o token comprometido, o pior caso é ver a ocupação da agenda.

Claim	Laboratório	Profissional
`aud`	`consultas-api:laboratorio`	`consultas-api`
`tipo` / `papel`	`servico` / `parceiro`	`usuario` / `profissional`

`scope`	`horarios:ler`	`consultas:ler`
		`consultas:escrever`
		`pacientes:ler`
		`profissionais:ler`

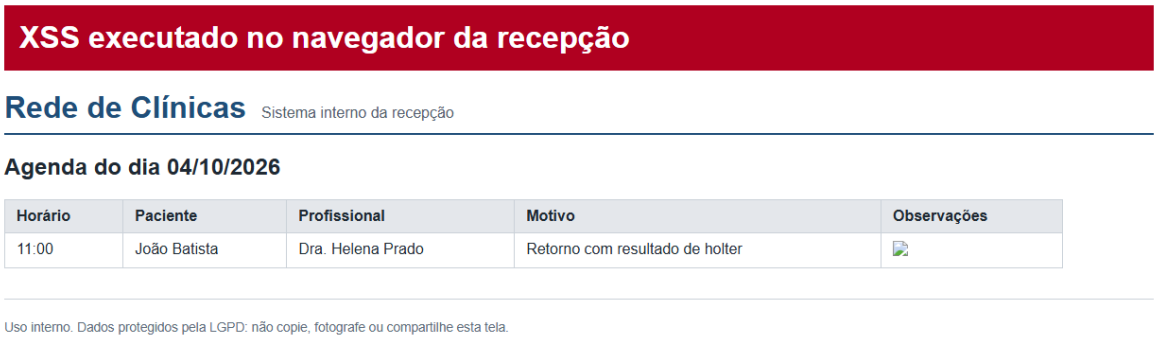
`amr`, `mfa`	ausentes	presentes
Validade	10 min	15 min

Evidência: `evidencias/ex07-m2m-escopos-claims.txt`.

Exercício 8 - Vulnerabilidades OWASP Top 10

As versões vulneráveis estão preservadas em `app/routes/inseguro.py` e `templates/agenda_inseguro.html`, montadas só com `CLINICA_MODO_DEMONSTRACAO_INSEGURA=true`, para o antes e depois poder ser refeito.

ID	Vulnerabilidade	OWASP	Onde (código)	Reprodução	
V1	BOLA em `GET /consultas/{id}`	A01	`inseguro.py:29-31`: confere token e escopo, mas carrega o objeto só pelo id	O paciente João lê o motivo clínico da paciente Maria	
V2	SQL injection na busca	A03	`inseguro.py:43`: f-string no SQL	`' OR '1'='1` lista todos os CPFs; `UNION` devolve os hashes de senha	
V3	XSS armazenado	A03	`agenda_inseguro.html:18`: `	safe`	Script executa no navegador da recepção
V4	Mass assignment	A04	`inseguro.py:62-71`: corpo `dict` espalhado sobre os campos do servidor	Consulta criada em nome de outro profissional, com trilha falsificada	
V5	Login sem limite, com enumeração e sem MFA	A07	`inseguro.py:91-106`	8 tentativas sem bloqueio; "Usuário não cadastrado" versus "Senha incorreta"; token de admin só com senha	
V6	Respostas sem cabeçalhos de segurança	A05	Router sem o middleware de cabeçalhos	Sem HSTS, XFO, XCTO e CSP	



Versão vulnerável: o script injetado executa

Evidência: `evidencias/ex08-reproducao-vulnerabilidades.txt`.

Exercício 9 - Correção de entrada e saída

Vulnerabilidade	Correção centralizada	Depois	
V1 BOLA	`AutenticacaoJWTMiddleware` (único ponto que valida o JWT, fail-closed) + `exigir_posse` + `GuardaAutorizacaoMiddleware` (bloqueia 2xx sem verificação de posse)	403, registrado na trilha de auditoria	

V2 SQL injection	Whitelist por regex no termo (`app/validators.py`) e enums para especialidade e status; repositório só com `select()`	422, sem refletir o payload	
V4 Mass assignment	Todas as entradas herdam `EntradaEstrita` com `extra="forbid"`; erro 422 sem eco do valor enviado	422 `extra_forbidden`	
V3 XSS	Autoescape do Jinja2, sem `	safe; `CSP` script-src 'self``	Payload exibido como texto

Endpoint não citado no Exercício 8 com o mesmo padrão: `GET /pacientes/{id}` permitia que um profissional lesse o cadastro de paciente de outro. Recebeu a mesma fábrica (`paciente_legivel`), assim como `PUT/DELETE /consultas/{id}` (`consulta_alteravel`), a busca de pacientes (filtrada pelo responsável) e a criação de consulta. Antes, a Dra. Helena lia a paciente do Dr. Marcos (200); depois, recebe 403.

Evidência: `evidencias/ex09-correcoes-antes-depois.txt` (mesmos payloads do Exercício 8).

Exercício 10 - Hardening de rede

- **CORS:** allowlist explícita em `CLINICA_CORS_ORIGENS`; o wildcard derruba a inicialização (`app/config.py`). Métodos e cabeçalhos também são enumerados.
- **Cabeçalhos** (`app/middlewares.py`, em toda resposta, inclusive 401, 404 e 429): `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, CSP, `Cache-Control: no-store`, e, após o ZAP, COOP, CORP, COEP e `Permissions-Policy`.

- **Rate limiting** (slowapi, por IP): 5/min no login e no MFA, 20/min em escrita, 60/min em leitura, 120/min como padrão. Um limite único erraria dos dois lados. Como limite por IP não para ataque distribuído, 5 falhas para o mesmo e-mail bloqueiam a conta por 15 minutos.

Evidência: `evidencias/ex10-cors-cabecalhos-rate-limit.txt` (6ª tentativa de login com 429 e `Retry-After`; leituras seguem liberadas).

Exercício 11 - Persistência segura

- SQLModel com `select()`, `where()` e `sessao.get()`: todo valor viaja como parâmetro ligado. Chamado direto com `' OR '1'='1`, o repositório gera `LIKE '%' || ? || '%'` e não encontra nenhuma linha.
- Sessão por requisição via dependência (`get_session` com `yield`, `SessaoDep`).
- Credenciais com `BaseSettings` e `.env`: `jwt_secret` como `SecretStr`, obrigatório e com no mínimo 32 caracteres (sem ele a aplicação não sobe); URL do banco vinda do ambiente; só o `.env.example` é entregue.

Evidência: `evidencias/ex11-persistencia-e-credenciais.txt`.

Exercício 12 - Pipeline DevSecOps

Fase do SDLC de cada ferramenta:

Tipo	Ferramenta	Fase	Por quê
SAST	Bandit, Semgrep	Commit/PR	Sem aplicação rodando, em segundos, aponta arquivo e linha; pega SQL concatenado e credencial no código

SCA	pip-audit, Dependabot	Build e semanal	Dependência fica vulnerável sem mudança no código
Testes de segurança	pytest (174), auditoria OpenAPI	Build	Falhas de autorização dependem do negócio e só teste escrito do threat model as encontra
DAST	ZAP passivo e autenticado	Homologação efêmera	Precisa da aplicação no ar com a configuração real; passivo para rodar em todo PR
IAST	Agente instrumentado	QA	Não implementado: sem agente gratuito para FastAPI (risco R-08)

Priorização (CVSS 3.1 e impacto de negócio):

ID	Vulnerabilidade	CVSS	Impacto de negócio	Prioridade
V-10	Contas de avaliação com hash no código	9.1	Credenciais publicadas dão acesso total se carregadas em produção	P0
V-07	Admin sem segundo fator	8.1	Uma senha vazada entrega toda a base	P0
V-03	SQL injection	7.6	Extração em massa de CPFs e hashes	P0
V-04	XSS armazenado	7.6	Atinge a recepção de toda a clínica	P0
V-06	Login sem limite e com enumeração	7.4	Tomada de conta	P0

V-01	BOLA em consultas	6.5	Dado de saúde de terceiros; sanção da LGPD	P0
V-05	Mass assignment	6.5	Agenda e trilha falsificadas	P1
V-09	CORS com wildcard	5.3	Reprovação em auditoria	P1
V-02, V-11, V-12, V-08	BOLA em pacientes, CSRF, eco no 422, cabeçalhos	4.2-4.3	Exposição limitada	P1/P2

Critério do security gate (`ci/security_gate.py`). Bloqueia o pipeline quando houver:

1. achado alto ou crítico (CVSS ≥ 7.0);
2. achado médio com CWE de controle de acesso (A01), injeção (A03) ou credencial no código;
3. vulnerabilidade conhecida em dependência, ou relatório ausente.

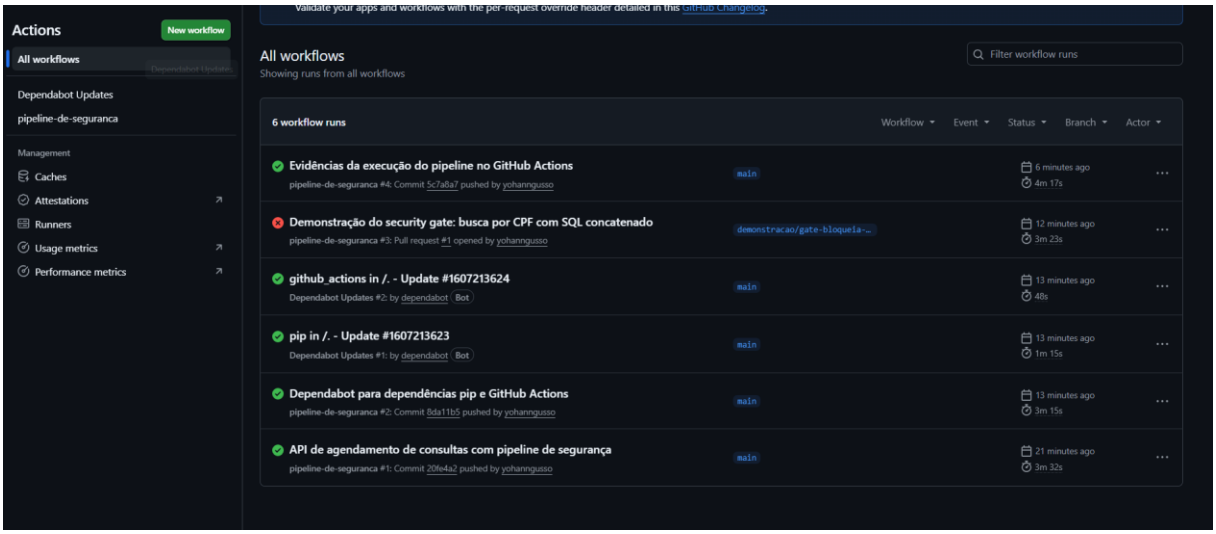
Exceções exigem arquivo, justificativa e validade (`ci/excecoes-seguranca.json`).

Justificativa: a BOLA do enunciado tem só 6.5 no CVSS, porque a fórmula não sabe que o dado é de saúde. Cortar só em 7.0 deixaria passar a falha mais grave do trabalho. As ferramentas também subestimam: o Bandit classifica SQL concatenado como médio. Bloquear todo médio pararia o time por achados de baixo impacto.

O gate em ação (execução local): a rodada 1 bloqueou os hashes das contas de avaliação no código (Semgrep, CWE-798), que foram movidos para `app/seed.py`, desligado por padrão. A rodada 3 bloqueou um `curl | python` no workflow e um cast sobre o token CSRF, ambos corrigidos. A versão final foi aprovada.

GitHub Actions (repositório privado `yohanngusso/consultas-api-dr2-at`; actions fixadas por SHA e imagens por digest):

Execução	Resultado
Push na `main`	5 jobs verdes (SAST, SCA, testes, DAST, gate)
PR #1 com SQL concatenado em `app/database.py`	Testes verdes, **gate bloqueou** (Semgrep alta e Bandit média com CWE-89); fechado sem merge



Histórico de execuções no GitHub Actions

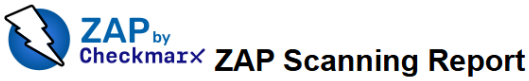
Testes de autorização expandidos: o teste do Exercício 6 virou uma suíte com um ou mais testes por ameaça (T-01 a T-22): BOLA, enumeração, token do laboratório fora do contrato, `alg=none`, assinatura forjada, token expirado, audiência trocada, admin sem MFA, mass assignment, CSRF.

Evidências: `evidencias/pipeline/` (Bandit, Semgrep e pip-audit em JSON), `evidencias/github-actions/` (logs dos jobs), `evidencias/ex12-ex13-suite-pytest.txt`.

Exercício 13 - Capstone

ZAP passivo (Docker, autenticado): `zap-baseline.py` na página da recepção e `zap-api-scan.py -S` sobre o OpenAPI.

Finding (rodada 1)	Risco	OWASP	Correção	Rodada 2
Absence of Anti-CSRF Tokens	Médio	A01	Token CSRF em cookie + campo oculto (`app/auth/csrf.py`)	Resolvido
COEP/COOP/CORP ausentes	Baixo	A05	Cabeçalhos adicionados	Resolvido
Permissions-Policy ausente	Baixo	A05	Cabeçalho adicionado	Resolvido
Cookie Poisoning	Info	A08	Cookie CSRF não é regravado com valor da requisição	Resolvido
Timestamp Disclosure (`/auth/eu`)	Baixo	A01	Falso positivo justificado em `.zap/rules.tsv`	Aceito



Site: http://host.docker.internal:8000

Generated on Sun, 4 Oct 2026 00:27:33

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	0
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 2xx	54 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 3xx	9 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 4xx	36 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type application/json	44 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type text/css	11 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type text/html	33 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with method GET	77 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with method POST	22 %
Info	Informational	http://host.docker.internal:8000	Count of total endpoints	9

ZAP baseline: rodada 2 sem alertas médios ou altos

Testes unitários com mocking (`tests/test_unitarios_mock.py`, 50 testes): regras de posse com objetos falsos, `exigir_posse` com carregador e auditoria mockados, `obter_principal` com repositório mockado, guarda de posse num app mínimo, bloqueio por identidade, expiração do MFA com relógio mockado e validadores com data fixa.

Auditoria OpenAPI (`ci/auditar_openapi.py`): verifica segurança nas rotas protegidas, ausência de campos internos nas respostas, `additionalProperties: false`, limites de tamanho e respostas 401/403/429 documentadas. Passou de 15 achados para 0; com o router de demonstração ligado, acusa 2 achados altos.

Rastreabilidade:

Ameaça	OWASP	Encontrada por	Correção	Evidência

T-01 BOLA	A01	Leitura de código (V1)	`exigir_posse` + guarda	`ex09`, `test_t01_*`
T-09 SQL injection	A03	Leitura de código, Bandit, Semgrep	Whitelist + `select()`	`ex09`, `ex11`, `test_t09_*`
T-10 XSS	A03	Leitura de código (V3)	Autoescape + CSP	telas 04 e 05
T-11 Mass assignment	A04	Leitura de código (V4)	`extra="forbid"`	`test_t11_*`
T-05/T-08/T-21 Autenticação	A07	Leitura de código (V5)	Rate limit, bloqueio, MFA, mensagem única	`ex10`, `test_t05_*`
T-13/T-14 Configuração	A05	V6 e ZAP	Cabeçalhos, CORS allowlist	ZAP rodada 2, `test_t13_*`
T-17 Credencial no código	A07	Semgrep	Seed separado, `SecretStr`	gate rodada 1 → final
T-19 CSRF	A01	ZAP	Token CSRF	ZAP rodada 2, `test_t19_*`
T-04 Laboratório	A01	Threat model	client_credentials, recorte, `aud`	`ex07`, `test_t04_*`

Riscos residuais:

ID	Risco	Bloqueia produção?
R-01	MFA simulado (código no log)	**Sim**
R-06	Router de demonstração e contas de avaliação no código	**Sim**, enquanto estiverem no build
R-05	SQLite sem criptografia em repouso	**Sim**
R-03	TLS terminado no proxy	Não, se o proxy aceitar só HTTPS
R-02	Rate limit em memória	Não em instância única
R-04	JWT sem revogação antes do `exp`	Não (15 min; desativação imediata)

R-07	CSP de `/docs` com `'unsafe-inline'`	Não, com docs desligado em produção
R-08	IAST não implementado	Não
R-10	Gate não obrigatório no merge (exige GitHub Pro)	Não, com revisão de PR

Veredito: liberação condicionada. Homologação aprovada: nenhum achado médio ou alto aberto, gate aprovado, todas as ameaças com mitigação e teste. A produção fica bloqueada até três coisas: substituir o MFA simulado por TOTP, gerar o build sem o código de demonstração e o seed, e usar banco com criptografia em repouso. Num serviço com dado de saúde, cada um desses riscos sozinho expõe a base inteira.